MIGHTY ADS

Complete Agency Management System

Functional Requirements • Workflows • Supabase Architecture • Database Logic • Security • Build Specification

A single implementation blueprint for Antigravity + Supabase

Includes screen-by-screen requirements, state transitions, SQL design patterns, RLS policies, Edge Functions, invoice and payroll logic, testing plan, and a master build prompt.

How to use this document

Give this document to Antigravity as the source of truth. Ask it to inspect the project, create the database migrations and security model first, and then implement the workflows. Do not ask it to generate every module in one untested code dump. The goal is one complete system, delivered through validated implementation phases.

Important: This version specifies the business logic and Supabase design in more detail than a feature overview. Exact legal/tax treatment, attendance policy, payroll rules, and invoice numbering should be confirmed before production use.

1. Product scope and principles

Build a responsive internal operations platform for Mighty Ads. Admin/Owner manages the agency; Managers coordinate work; employees execute assigned tasks. Client and billing records are connected to tasks and deliverables.

• Single organization for the initial release, but include organization_id in the data model if there is a realistic chance of selling this as SaaS later.

• Desktop-first responsive UI, usable on mobile.

• Use real database operations, not mock-only cards or inert buttons.

• Use server-side timestamps and database constraints for authoritative records.

• Keep attendance time, break time, task time, and payroll calculation separate.

• Sensitive operations (role changes, invoice issuance, payroll approval, time corrections) require server-side authorization and audit records.

Role	Can do	Must not do by default
Admin / Owner	Manage all users, settings, clients, attendance corrections, invoices, payroll, reports.	Nothing outside organization security boundaries.
Manager	Manage authorized clients/tasks, assign work, review submissions, see team workload.	Change roles, view salary details, issue invoices or change payroll unless separately granted.
Employee	See own profile, attendance, assigned tasks, timers, notifications; submit work and leave requests.	Read other employees' private records or salary; alter approved time records.
Payroll/Finance (optional)	View payroll/invoice modules if explicitly granted.	Manage auth users or unrelated task data without permission.

2. Application map and navigation

Area	Pages	Primary actions
Overview	Admin / Manager / Employee dashboard	KPIs, shortcuts, alerts, current work status.
Team	Employee list, employee detail, roles	Create/invite, edit profile, deactivate, role assignment.
Attendance	Today, history, exceptions, leave	Check in, lunch start/resume, check out, correction review.
Tasks	List, Kanban, task detail, review queue	Create, assign, comment, attach, start/pause timer, submit/review.
Clients	Client list, profile, services, deliverables	Create client, define retainer, set monthly outputs, link tasks.
Billing	Invoice list, invoice detail, payments, bulk run	Generate individual/all, preview, issue PDF, record payment.
Payroll	Salary records, monthly run, payslips	Calculate draft, review, approve, mark paid.
Reports	Attendance, task time, deliverables, billing, payroll	Filter by date, employee, client, status; export.
Settings	Company profile, invoice settings, attendance rules, permissions	Configure timezone, working hours, numbering, tax, policies.
Audit	Privileged action history	Filter and inspect immutable audit entries.

3. Core workflow specifications

3.1 Login and account provisioning

1. Admin creates/invites an employee through a trusted server-side process using Supabase Auth Admin APIs (Edge Function or secure server environment).

2. The user receives an invitation or temporary sign-in flow; on first login, require password setup if using password authentication.

3. A profiles row is created/linked to auth.users.id. The user cannot choose or self-promote their role.

4. After login, load the profile and route to the role-appropriate dashboard.

5. If account is inactive, deny application access and sign out or show an account-disabled message.

6. Password reset uses Supabase Auth's supported reset flow; never store raw passwords in application tables.

3.2 Attendance and lunch state machine

Current state	Action	Next state	Rules
Not checked in	Check In	Working	Create one attendance record for local business date; server timestamp.
Working	Start Lunch	On Lunch	Create open break interval; only one open break.
On Lunch	Resume Work	Working	Close break interval using server timestamp.
Working	Check Out	Checked Out	No open break or timer unless policy explicitly auto-pauses/stops it.
On Lunch	Check Out	Checked Out	Close lunch interval, flag automatic closure if employee checked out during lunch.
Checked Out	Check In again	Blocked by default	Use Admin correction or configured split-shift feature.

• Check-in should be a visible action after login by default. Auto-attendance on login is an optional policy because logging in is not always the same as starting work.

• Use Asia/Kolkata for display and work-date grouping; store timestamps as timestamptz (UTC internally).

• Net work duration = checkout – check-in – sum of recorded break intervals. Validate interval order and avoid double subtraction.

• Missed checkout: mark an exception, notify Admin, and allow a reasoned correction with old value/new value/actor in audit log.

• Do not infer absence from a single login event. Attendance rules must account for approved leave, holidays, and configured work schedule.

3.3 Leave workflow

1. Employee submits leave type, start/end dates, reason, and optional attachment.

2. Status becomes Pending; manager/admin receives notification.

3. Authorized reviewer approves or rejects with a comment.

4. Approved leave appears in attendance summaries and should not be counted as absence.

5. Changes after approval must be audited.

3.4 Task assignment and review

1. Manager creates task and selects client (optional), assignee, title, instructions, priority, deadline, estimated duration, and attachments.

2. Server validates that assigner can access the client and assign the selected employee.

3. Task is saved as Assigned; a notification row is created in the same database transaction where practical.

4. Employee opens task, reviews deadline and assignment source, then starts work.

5. Employee submits work for review with links/files/comments; status becomes In Review.

6. Reviewer approves as Completed or requests Revision Required with feedback.

7. Revision returns to In Progress when employee resumes. Preserve status history and time logs.

8. Task reassignment records previous assignee, new assignee, actor, and timestamp.

Transition	Who can trigger	Validation
Assigned → In Progress	Assignee or authorized manager	No conflicting active timer; task is not cancelled/completed.
In Progress → In Review	Assignee or authorized manager	Optional required completion note/file based on task type.
In Review → Completed	Reviewer/manager	Record reviewer and approval time.
In Review → Revision Required	Reviewer/manager	Feedback/comment required.
Revision Required → In Progress	Assignee	New work session may start.
Any → Cancelled	Authorized manager/admin	Reason required; stop/close active timer safely.

3.5 Persistent task timer logic

1. Start Timer calls a server-side function that checks the user identity, task access, task status, and absence of another active timer.

2. Insert a time entry with started_at = database/server timestamp and ended_at = NULL.

3. Use a partial unique index or equivalent database guard to allow at most one open time entry per employee.

4. Pause/Stop sets ended_at using server time and computes duration; the UI timer display is derived from stored start time plus elapsed time.

5. On refresh/re-login, query the user's open time entry and reconstruct the timer display.

6. Do not trust a duration supplied by the browser. If Admin edits an entry, preserve original values and log the correction.

7. Define what happens at lunch/check-out: recommended default is warn and require the employee to pause the timer; do not silently stop work timers.

4. Client CRM and deliverable logic

Client profile fields

• Client/company name, contact person, email, phone, billing address, GSTIN (optional), billing notes, status, assigned account manager.

• Contract start/end date, billing day, payment terms, recurring package amount, currency (INR default).

• Services: name, recurring or one-time, rate, tax configuration, active dates, billing description.

• Deliverables: period, deliverable type, target quantity, completed quantity, owner, status, evidence/link.

• Related tasks, invoice history, payment history, notes, and files.

Deliverable workflow

1. At the start of a billing period, create or materialize the expected deliverables from the client's active service template.

2. Managers create tasks linked to the client and optionally link each task to a deliverable.

3. When a task is approved, authorized logic increments the deliverable completion count or records a completion event; prevent double-counting if task status is toggled.

4. Client summary shows target vs completed deliverables and outstanding work.

5. If the package changes mid-month, keep a dated service history; do not retroactively change issued invoices.

Suggested client lifecycle

Lead/Prospect (optional) → Active → Paused → Ended. Only Active clients with billable services are included in bulk invoice generation by default. Paused or ended clients require explicit override.

5. Invoice generation and payment logic

Invoice run — individual

1. Admin selects client and billing period.

2. Backend loads active billable services for the period and approved one-time charges.

3. Show a preview of line items, discounts, configured taxes, totals, due date, and billing address.

4. Admin confirms; server validates permissions and checks for an existing invoice for that client/period.

5. Create invoice and items atomically, assign unique invoice number, generate/store PDF, and return invoice detail.

6. Issue/send invoice only after review; sending email is a separate integration requiring provider credentials.

Invoice run — all

1. Admin selects billing period and clicks Generate Invoice All.

2. Backend fetches eligible active clients with billable services and prepares a preview/count.

3. A background/batch operation processes each client independently and idempotently.

4. For each client, check unique client+period constraint; create invoice/items if none exists; otherwise skip and report reason.

5. Generate PDFs and record status for each invoice; retry transient failures safely.

6. Return summary: created, skipped, failed, and links to invoices. One failed client must not corrupt other invoices.

Invoice calculation rules

Field	Recommended rule
Line subtotal	quantity × unit price, rounded consistently to configured currency precision.
Discount	Explicit fixed or percentage discount; define whether applied before tax.
Tax	Configurable per service/client and confirmed by finance. Do not hard-code one tax rate for all clients.
Grand total	Sum of line totals after discount and tax according to configured policy.
Payments	Store payments as separate records; outstanding = invoice total – sum(valid recorded payments).
Status	Draft, Issued, Partially Paid, Paid, Overdue, Cancelled. Derive overdue from due date and balance where appropriate.
Duplicate prevention	Unique client_id + billing_period + invoice type for recurring invoices; unique invoice number.
Issued invoice edits	Do not silently overwrite. Use controlled revision, cancellation, or credit note policy.

PDF and numbering

• Invoice number format configurable, e.g. MA-YYYY-0001; generation must be concurrency-safe.

• PDF includes agency identity, invoice number/date, client billing details, period, line items, taxes, totals, payment instructions, and terms.

• Store PDF in a private Supabase Storage bucket; use authorized download or short-lived signed URLs.

• Keep draft generation separate from issuance; issuing should freeze the commercial snapshot used on the invoice.

Payment recording

Record payment date, amount, method, reference, and notes. Validate amount > 0; do not let total valid payments exceed invoice balance unless overpayment/credit handling is explicitly supported. Partial payments update outstanding balance and payment status.

6. Salary and payroll logic

Payroll workflow

1. Admin selects payroll month and employee group.

2. Backend loads salary terms effective for that period, approved leave, attendance summaries, and approved adjustments.

3. Generate draft payroll rows; show gross earnings, bonuses, advances, deductions, and net payable.

4. Admin reviews and approves; approval freezes the calculation snapshot.

5. Generate payslip PDF and record payment date/reference/status.

6. Corrections after approval require a revision/adjustment and audit log.

• Do not automatically deduct salary merely because task-tracked time is low.

• Attendance-based deductions, overtime, holidays, leave, and statutory payroll treatment must be configurable and reviewed against applicable Indian employment and tax requirements.

• Employees can see their own payslips only if policy allows; managers do not automatically receive payroll access.

• Salary records must not be exposed in general employee profile queries.

7. Supabase architecture

Supabase component	How it is used
Auth	Email/password or invitations, sessions, reset flow. No public signup.
PostgreSQL	Source of truth for profiles, attendance, tasks, timers, clients, invoices, payments, payroll.
RLS	Enforce row-level access on all exposed tables; frontend visibility is not security.
SQL functions / RPC	Atomic state transitions such as timer start/stop, attendance transitions, invoice creation helpers where appropriate.
Edge Functions	Privileged user invitations, invoice batch orchestration, PDF/email workflows, payroll approval; validate JWT and role.
Realtime	Subscribe to authorized task/notification changes. RLS still governs access.
Storage	Private buckets for attachments, invoices, payslips; policies or signed URLs.
Cron	Optional scheduled reminders and invoice run preparation; must be idempotent and observable.
Netlify	Frontend deployment; environment variables configured separately for preview and production.

Recommended database naming and conventions

• Use UUID primary keys, created_at/updated_at timestamptz, foreign keys, and indexes for common filters.

• Use enums or check constraints for finite statuses; validate transitions in server functions, not just UI.

• Use numeric/decimal columns for currency, never floating-point values for money.

• Store durations as integer seconds or derive them from timestamps consistently.

• Use organization_id in business tables if multi-tenant SaaS is planned; enforce organization isolation in every RLS policy.

Core table inventory

Table	Key data / purpose
profiles	auth user id, employee code, name, role, department, employment status; avoid broad salary exposure.
attendance	employee_id, work_date, check_in_at, check_out_at, status, correction flag.
attendance_breaks	attendance_id, break_type, started_at, ended_at.
leave_requests	employee_id, dates, leave_type, reason, status, reviewer.
clients	client identity, billing address, GSTIN, account manager, lifecycle.
client_services	client_id, service, recurring flag, rate, tax config, active dates.
client_deliverables	client_id, period, type, target/completed, owner, status.
tasks	client_id, assigner_id, assignee_id, title, description, priority, deadline, status.
task_status_history	task_id, from_status, to_status, actor, reason, created_at.
task_comments	task_id, author_id, body, created_at.
task_attachments	task_id, storage path, uploader_id, metadata.
task_time_entries	task_id, employee_id, started_at, ended_at, duration_seconds, correction metadata.
notifications	recipient_id, type, message, related entity, read_at.
invoices	client_id, billing period, number, issue/due date, status, frozen totals snapshot.
invoice_items	invoice_id, description, quantity, unit_price, discount, tax, total.
invoice_payments	invoice_id, amount, date, method, reference.
salary_terms	employee_id, effective dates, fixed salary and policy reference; restrict access.
salary_records	employee_id, payroll period, calculation snapshot, net payable, status.
audit_logs	actor, action, entity, before/after metadata where appropriate, timestamp.

8. Supabase SQL implementation patterns

The snippets below are design patterns to guide Antigravity. They must be adapted to the final schema, tested in a staging project, and applied as version-controlled migrations. They are not a substitute for reviewing the full RLS model.

8.1 Profiles linked to Auth

```
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  employee_code text unique not null,
  full_name text not null,
  role text not null check (role in ('admin','manager','employee','payroll')),
  department text,
  is_active boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
alter table public.profiles enable row level security;
```

8.2 Security helper functions

```
create or replace function public.current_app_role()
returns text
language sql stable security definer
set search_path = public
as $$
  select role from public.profiles
  where id = (select auth.uid()) and is_active = true
$$;

create or replace function public.is_admin()
returns boolean
language sql stable security definer
set search_path = public
as $$
  select coalesce(public.current_app_role() = 'admin', false)
$$;
```

Security note: security-definer functions require careful ownership, fixed search_path, minimal grants, and testing to avoid privilege escalation. Do not make helper functions executable by everyone unless intended.

8.3 Example profile policy

```
create policy "read own profile or admin"
on public.profiles for select
to authenticated
using (id = (select auth.uid()) or public.is_admin());

create policy "admin updates profiles"
on public.profiles for update
to authenticated
using (public.is_admin())
with check (public.is_admin());
```

This example is intentionally narrow. A production schema needs table-by-table SELECT/INSERT/UPDATE/DELETE policies and must avoid letting a user update their own role or active status. For sensitive fields such as salary, use a separate restricted table instead of placing them in a profile row every employee can read.

8.4 One active task timer per employee

```
create unique index one_open_timer_per_employee
on public.task_time_entries (employee_id)
where ended_at is null;
```

Start/stop should be performed by a transaction-safe SQL RPC or Edge Function. Start validates task access and status, then inserts an open entry. Stop validates ownership, sets ended_at = now(), and computes duration from server timestamps. Handle unique-index violations as a friendly 'another timer is running' response.

8.5 Invoice uniqueness

```
-- Example for recurring monthly invoices; use a normalized period_start date.
create unique index unique_recurring_invoice_per_client_period
on public.invoices (client_id, period_start, invoice_type)
where invoice_type = 'recurring' and status <> 'cancelled';
```

Choose the final cancellation/reissue policy before relying on this exact constraint. Invoice numbering must use a sequence or atomic server-side allocator, not a client-generated count.

8.6 Attendance constraints and transitions

• Unique attendance record per employee and work_date for a single-shift policy; if split shifts are needed, model attendance_sessions separately.

• Break rows must belong to the correct attendance record; only one open break per attendance session.

• A server-side transition function validates current state and actor before changing check-in, lunch, resume, or check-out.

• Never allow browser-supplied timestamps to overwrite authoritative attendance timestamps without an explicit correction workflow.

8.7 RLS policy design matrix

Data	Employee	Manager	Admin
Profiles	Read own safe profile fields	Read permitted team profiles	Full org access
Attendance	Read own; create own valid transitions	Read authorized team; corrections only if granted	Read/correct with audit
Tasks	Read assigned; update allowed task fields	Create/assign/review authorized tasks	Full access
Time entries	Read own; start/stop own timer	Read authorized team entries if policy allows	Read/correct with audit
Clients/services	Only authorized client data needed for tasks	Manage authorized clients	Full access
Invoices/payments	No access by default	No access unless granted	Full access
Salary records	Own payslip only if policy allows	No access by default	Full access
Audit logs	No access by default	Limited audit if granted	Full access

9. Edge Functions and backend endpoints

Function / RPC	Input	Expected behavior
admin-invite-employee	Email, profile fields, role	Verify caller is Admin; invite/create Auth user; create profile; never accept arbitrary privilege from untrusted client.
attendance-transition	Action: check_in/start_lunch/resume/check_out	Use authenticated identity, server time, validate state and work date, write audit event where needed.
task-start-timer	Task ID	Check task authorization/status; enforce one active timer; create entry with server timestamp.
task-stop-timer	Time entry ID	Check owner/admin; close entry with server timestamp; compute duration.
task-submit-review	Task ID, note/attachments	Validate assignee; change status and notify reviewer.
task-review	Task ID, approve/revision, comment	Validate reviewer; update status and history; notify assignee.
generate-invoice	Client ID, period, preview confirmation	Validate Admin; calculate server-side; enforce uniqueness; save invoice snapshot.
bulk-generate-invoices	Period, optional client filters	Admin only; process idempotently in batches; return created/skipped/failed summary.
generate-invoice-pdf	Invoice ID	Authorize access; render PDF; save to private storage; return authorized download reference.
record-invoice-payment	Invoice ID, amount, date, method, reference	Authorize finance/admin; validate amount; update derived status safely.
calculate-payroll-draft	Payroll period, employee IDs	Payroll/Admin only; calculate draft using configured rules and effective salary terms.
approve-payroll	Payroll run ID	Authorized approver only; freeze calculation snapshot; audit approval.
mark-salary-paid	Salary record ID, date, reference	Authorized payroll user; update status and audit event.

Do not create an Edge Function for every trivial read. Use direct Supabase queries where RLS safely covers the operation; use server functions/RPC for privileged actions, state transitions, atomic financial operations, and workflows needing secrets.

10. Notifications and automation

Trigger	Notification
Task assigned/reassigned	Assignee; include task title, deadline, assigner.
Deadline approaching	Assignee and optionally manager; configurable intervals.
Task submitted for review	Reviewer/manager.
Revision requested / approved	Assignee.
Leave submitted / decision	Reviewer and employee.
Missed checkout exception	Employee and Admin.
Invoice generated / due / overdue	Admin/finance; client email only if email sending is configured.
Payroll ready for approval	Authorized payroll approver.

• In-app notifications are the baseline; email/WhatsApp notifications require provider credentials, consent, templates, and delivery/error handling.

• Use idempotency keys or unique event references so retries do not create duplicate notifications.

• Scheduled reminders should run through Cron or a scheduled Edge Function and be logged; avoid relying on a browser tab being open.

11. Frontend implementation rules

• Use route guards for user experience, but rely on RLS/server checks for actual authorization.

• Create reusable components: data table, filters, status pill, modal form, confirmation dialog, empty state, loading state, error banner, date range picker.

• Every form needs validation, disabled/submitting state, success/error feedback, and safe retry behavior.

• Do not show a success toast until the server confirms success.

• Do not keep authoritative attendance or timer data only in React state.

• Use pagination for tables and server-side filtering for larger datasets.

• Use Asia/Kolkata display formatting consistently; store timestamptz values and convert for display.

• Use INR formatting and decimal/numeric calculations for money.

• Provide accessible labels, keyboard navigation, and responsive mobile layouts.

12. Testing and acceptance criteria

Area	Required tests
Auth	Invitation, login, reset password, disabled user, session expiry, role escalation attempt.
RLS	Employee cannot read another employee's private rows; manager cannot read payroll by default; unauthorized API requests fail.
Attendance	Duplicate check-in blocked; lunch transition valid; invalid transitions rejected; missed checkout flagged; timezone boundary tested.
Timers	Refresh recovery; repeated start click; concurrent start requests; pause/stop; invalid negative duration blocked; admin correction audited.
Tasks	Assignment notification; deadline display; revision loop; reassignment history; cancelled task timer handling.
Deliverables	Approved task increments completion once; toggling/retries do not double count.
Invoices	Totals reconcile; unique invoice number; duplicate period blocked; bulk partial failure reported; PDF snapshot matches invoice.
Payments	Partial payment balance; full payment status; invalid amount rejected; audit trail.
Payroll	Draft vs approved snapshot; salary data restricted; payslip permission; correction after approval audited.
Deployment	Secrets absent from bundle/repo; staging test; database backup/recovery plan; error logs and rollback steps.

Production gate: do not use real payroll or issue real client invoices until an authorized person validates the tax configuration, invoice numbering, payroll rules, permissions, and PDF outputs.

13. Build order for Antigravity

Stage	Implementation output	Definition of done
0. Inspect & plan	Repo audit, architecture, schema ERD, migrations plan, environment docs.	No conflicting framework assumptions; schema and permissions reviewed.
1. Foundation	React app, Supabase client, Auth, roles, profiles, RLS, app shell.	Protected routes; role checks; unauthorized queries fail.
2. Attendance	Check-in/out, breaks, leave, exceptions, admin correction audit.	State machine tests pass.
3. Tasks & timers	Task CRUD, assignment, comments, review, notifications, persistent timer.	Timer constraints and review workflow pass.
4. Clients	Client profiles, services, contracts, deliverable templates and progress.	Client and task links work.
5. Billing	Invoice preview, individual/bulk run, PDF, payments, duplicate guard.	Financial totals and bulk retry tests pass.
6. Payroll	Salary terms, draft calculation, approval, payslips, payment status.	Role restrictions and calculation tests pass.
7. Reports & deploy	Dashboards, filters, exports, audit viewer, Netlify deployment.	Acceptance suite and security review pass.

14. Master Antigravity prompt

```
You are the senior full-stack engineer building Mighty Ads Agency Management System. Treat the accompanying Complete Software Specification as

IMPORTANT WORK RULES:
1. Inspect the existing repository and report its current state before editing.
2. Create a written implementation plan, relational schema/ERD, SQL migrations, and table-by-table RLS matrix before building the UI.
3. Implement real end-to-end features; no mock-only dashboards or inert buttons.
4. Keep secrets out of frontend code. Never expose the service-role key.
5. Use server timestamps and database constraints for attendance, breaks, task timers, invoice numbering, and billing-period uniqueness.
6. Separate attendance time, break time, task time, and payroll calculations.
7. Protect roles and sensitive data at the database/backend layer, not just by hiding UI controls.
8. Add automated tests for state transitions, authorization, timer concurrency, invoice idempotency, calculations, and payroll permissions.
9. Complete and validate each stage before moving to the next. Keep migrations, setup instructions, .env.example, seed/demo data, and deploymen
10. Do not claim an integration or feature is complete until it has been exercised successfully. If provider credentials are missing, document

MODULES:
- Admin-created employee accounts, profiles, roles, password reset, deactivation.
- Role-based Admin, Manager, Employee, and optional Payroll dashboards.
- Attendance check-in/check-out, lunch start/resume, leave, missed checkout exception and audited correction.
- Task assignment, client link, priority, deadline, comments, attachments, statuses Assigned/In Progress/In Review/Revision Required/Completed/
- In-app notifications and real-time updates.
- Persistent task timer with server timestamps, pause/resume/stop, one active timer per employee, refresh recovery, audited corrections.
- Client profiles, services, recurring retainers, one-time charges, contracts, deliverable targets and completion.
- Invoice generator: Generate Invoice Individually and Generate Invoice All, preview, unique numbering, client-period duplicate prevention, ite
- Salary/payroll: effective salary terms, draft calculation, approved adjustments, approval snapshot, payslips, restricted permissions.
- Dashboards, reports, audit logs, settings, responsive UI.

SUPABASE REQUIREMENTS:
- Create versioned SQL migrations, foreign keys, constraints, indexes, RLS for every exposed table, and a documented policy matrix.
- Use safe server-side functions/RPC for privileged operations and atomic state transitions.
- Validate authenticated identity and role in every privileged Edge Function.
- Use private Storage for invoices, payslips, and attachments.
- Include organization_id isolation if multi-tenant SaaS is planned; otherwise document the single-organization assumption.
- Use numeric/decimal for money and timestamptz for timestamps; display business times in Asia/Kolkata.

IMPLEMENTATION STAGES:
0 Inspect repo and propose schema/security model.
1 Auth, profiles, roles, RLS, app shell.
2 Attendance, lunch, leave, exception handling.
3 Tasks, notifications, review workflow, timers.
4 Clients, services, deliverables.
5 Individual/bulk invoices, PDFs, payments.
6 Payroll, payslips, reports, audit logs.
7 QA, security checks, deployment.

Start with Stage 0. Return the proposed schema, RLS matrix, migration plan, and risks for review. Then implement Stage 1. Do not skip stages or
```

15. Operational decisions to confirm before production

• Attendance policy: auto-check-in on login or explicit Check In button? Recommended: explicit Check In by default.

• Work schedule: standard hours, weekly offs, holidays, grace period, and leave policy.

• Timer policy: what happens if an employee logs out, starts lunch, or checks out while a timer is active?

• Invoice policy: numbering format, billing day, due date, discounts, tax treatment, cancellation/reissue procedure.

• Payroll policy: salary proration, approved deductions, overtime, advance recovery, payslip visibility.

• Notifications: in-app only for MVP, or email/WhatsApp as well? Provider accounts and consent requirements.

• Future SaaS: will other agencies sign up? If yes, organization_id and tenant isolation are mandatory from the start.